

SEEDX Emulator: A Scalable and High-Fidelity Platform for Internet Experimentation

Abstract

Existing Internet experimentation platforms are either prohibitively expensive or unable to simultaneously support realistic protocol execution, real-world topology reproduction, and large-scale experiments. To address this gap, we present **SEEDX Emulator**, a programmable Internet emulation platform for research. By decoupling composition from execution and adopting a layered design that separates topology, routing, and services, SEEDX provides a flexible, reusable, and execution-agnostic environment for Internet experimentation.

To achieve *fidelity*, *scalability*, and *extensibility*, SEEDX further integrates a real-world-topology construction pipeline, a BGP-aware subgraph generation algorithm, and a system-level optimization mechanism. Together, these designs enable SEEDX to preserve the key structural and routing semantics of the Internet, support the rapid integration of diverse services and network architectures, and mitigate RTNL contention caused by large-scale route installation, thereby enabling stable single-machine emulation with 10K+ routing nodes. Our case studies further demonstrate that SEEDX can support realistic experiments across control-plane security, clean-slate architectures, blockchain, CDN, and cybersecurity education, showing that it is not only a high-fidelity and scalable Internet emulation platform, but also a practical platform for distributed-systems research.

1 Introduction

Conducting large-scale Internet experiments has long been a challenging problem. There are two typical solutions to this problem. A common approach is to use a dedicated infrastructure, such as cyber ranges, consisting of many physical machines or virtual machines. The US National Science Foundation has funded some of such infrastructures, such as the DETER project [3]. Many companies and organization also offer cyber range as a service. These infrastructures are quite expensive to build and maintain. Based on a report [18], a cyber range costs \$336K to \$576K for the first year, and \$70K to \$100K for each consecutive year.

Another approach is to use a simulator/emulator to simulate or emulate the behaviors of a large network. Existing works include Common Open Research Emulator (CORE) [4], NS-3 [17], GNS-3 [24], SEED [21], Greybox [42], NetSim (commercial) [33], OPNET (commercial) [45], etc. These simulators/emulators can run on a single or a small number of computers, making them much more affordable than the infrastructure approach.

This work presents SEEDX, a high-fidelity Internet emulator for networking research. SEEDX builds upon the architectural foundations of the SEED emulator [21], which introduced a container-based environment primarily optimized for cybersecurity education. While the original SEED platform enabled the recreation of classic attack scenarios—such as BGP prefix hijacking and the Morris worm—its design was constrained by the requirements of a controlled classroom setting.

However, the requirements of pedagogical emulation and scientific research differ substantially. Educational tools prioritize feature richness and ease of deployment; in contrast, research-grade emulators demand extreme fidelity, massive scale, and programmatic extensibility. While a simplified topology suffices for illustrating the mechanics of a DNS spoofing attack, it is inadequate for rigorous scientific inquiry. To evaluate next-generation routing architectures or the resilience of distributed systems, researchers require an environment that mirrors real-world operational complexity—one capable of ingesting massive CAIDA AS-relationship datasets and scaling to tens of thousands of nodes.

Achieving high fidelity at this scale introduces fundamental design and system-level challenges. At the design level, research workflows are highly heterogeneous, necessitating an extensible orchestration layer that allows for fine-grained customization of nodes and networks. At the system level, the sheer density of the global Internet, comprising over 75,000 Autonomous Systems, pushes modern operating systems to their limits. When thousands of containerized routers concurrently update forwarding tables on shared hardware, kernel-level bottlenecks, such as Linux Routing Netlink (RTNL) lock

contention, significantly distort protocol convergence and temporal accuracy. Transforming an educational sandbox into a high-precision scientific instrument requires identifying and mitigating these deep-seated kernel limitations.

Contributions. In this paper, we introduce the architecture of SEEDX and describe our approach to achieve high-fidelity and large-scale Internet experimentation. We make the following contributions: (1) **An extensible architecture:** We present an emulation architecture that uses a layered composition design and decouples composition from execution, enabling clean extension of emulator functionality. (2) **Production-grade protocol and service integration:** Unlike traditional simulators that rely on abstract models, SEEDX runs production-grade software directly. We demonstrate its extensibility by integrating complex distributed systems and services, including Ethereum Proof-of-Stake (PoS) consensus mechanisms, large-scale BGP routing suites (BIRD), and CDN services. (3) **Large-scale emulation:** We characterize the performance degradation caused by Linux Routing Netlink (RTNL) lock contention in large-scale emulation, and propose mitigation mechanisms for this bottleneck. Using our scheme, emulation on a single Linux server with 384 threads and 1 TB of RAM can reach 10,000 routing nodes while retaining realistic routing dynamics and convergence behavior. (4) **Distributed emulation:** We add support for Kubernetes to achieve distributed emulation, which allows a cluster of computers to conduct emulation jointly, enabling large-scale emulation. (5) **Emulation using real data:** We design an automated topology-generation method that extracts Internet topology information and inter-AS peering relationships from CAIDA datasets and translates them into deployable emulator abstractions that is used in the emulation.

2 Related Work

A high-fidelity Internet emulator should cover the following three properties: (1) network emulation, (2) Internet infrastructure emulation, (3) service-layer emulation. The network emulation part covers hosts, routers, networks, and routing. This part is what most network emulators/simulators have achieved well, including NS-3 [17], Mininet [28], Containernet [38], Common Open Research Emulator (CORE) [4], GNS-3 [24], NetSim (commercial) [33], OPNET (commercial) [45], Kathara [10], ContainerLab [20], etc. Large-scale cybersecurity testbeds, such as DeterLab [7], also excel in this domain by providing high-fidelity, bare-metal network topologies. SEEDX supports standard network-emulation functionality; our main distinctions lie instead in Internet-infrastructure and service-layer emulation.

The second capability is Internet-infrastructure emulation: modeling the routing substrate of the Internet, including stub and transit ASes, Internet exchanges, BGP peering, and intra-domain routing such as OSPF. Since BGP (IBGP and EBGP)

is the core of this infrastructure, a good emulator should allow users to easily set up and configure real BGP software and peering, including capturing the business relationship of the peers (such as the provider-customer, and peer-to-peer relationships). While other network emulators like Mininet and powerful physical testbeds like DeterLab allow users to run actual routing daemons to achieve this, doing so requires labor-intensive, node-by-node manual configuration or custom scripting. Some of the existing work that covers this part include Greybox [42] and Mini-Internet project [25], but the approach that we are taking is very different and advantageous, especially on how the emulation is built and on the support of the BGP routing, peering, and configuration.

The third feature is the service-layer (or application-layer) emulation. One thing that makes the Internet interesting is the applications running on top of it. A good Internet emulator should be able to emulate these services, run actual software that users can interact with, from simple services such as Web and DHCP, to those involving their own infrastructure, such as DNS, Blockchain, Darknet, Botnet, Content Delivery Network (CDN), and Public-Key Infrastructure (PKI). Setting up these infrastructures inside an Internet emulator is non-trivial. In most existing work, deploying such services requires researchers to provision baseline OS images and manually orchestrate the installation of complex, distributed software across the topology. A good emulator should reduce this complexity for users. This is the part where **the existing emulators lack and our emulator really shines**. As we will see from Section 3, effectively supporting the service-layer emulation is an important and unique feature of our design.

More comparison with representative work. While the comparison above gives a general comparison with the existing work, we give a more detailed comparison with some of the representative work widely used by the network research community. We first compare our work with NS3 and Mininet. Both systems focus primarily on control-plane emulation rather than service-layer deployment. To run real applications on these emulators, users have to do it themselves, but NS3 and Mininet are fundamentally inadequate for this task. NS3 requires modification of the application, while Mininet, which is namespace-based emulation, does not separate filesystems, so running multiple instances of the same software on the same OS is quite difficult.

To overcome the filesystem isolation limitations of namespace-based emulation, extensions such as Containernet [38] have integrated Docker container support directly into the Mininet API. While this successfully isolates application state, it fundamentally fails to address the execution scalability bottleneck. Containernet remains architecturally tethered to a centralized execution model, forcing all Docker containers, the Open vSwitch data plane, and the controller to share a single physical host’s resources.

The SEEDX Emulator resolves this by strictly decoupling

the emulation construction and the physical execution. The construction of the emulation—such as defining topology and configuring system—is agnostic to the physical execution environment; translating the emulation to the execution environment is done by a separate module called compiler. Because of this design, extending the Docker-based emulation to Kubernetes-based emulation only requires a new compiler, which took less than a month to develop. If new execution environment emerges in the future, similar extension can be implemented in a short period of time.

Emulators for production-grade fidelity. There are a number of emulators that prioritize the high-fidelity validation of vendor-specific hardware and protocol implementations, including KNE (Kubernetes Network Emulation) [35], Containerlab [20], Cisco Modeling Lab [15], and NVIDIA Air [34]. They are designed to run real, containerized or virtualized Network Operating Systems (NOS) from vendors like Arista, Nokia, Juniper, and Cisco. These systems target product-level fidelity by running vendor network operating systems and closely approximating deployment environments. The SEEDX emulator and most emulators mentioned in this section on the other hand, focus on holistic study of the Internet as a complex system, so conceptual realism, using a lightweight container that behaves like a real router, is often sufficient, instead of using vendor-heavy images. Such a sacrifice makes it possible to emulate a large-scale network, which is hard to achieve by those production-focused emulators.

3 System Architecture

The design of the SEEDX Emulator is driven by three primary objectives: scalability, extensibility, and usability for both researchers and educators. To achieve these goals, our architecture takes two important design decisions. First, the architecture fundamentally decouples the logical definition of the emulated environment from its underlying physical execution infrastructure, so the composition of the emulation is completely execution agnostic. Second, the composition employs a layered design, similar to the layered image composition method utilized by most image editing software such as Photoshop.

3.1 Decoupling Composition from Execution

As illustrated in Figure 1, the complete emulation lifecycle consists of four distinct phases: *composition*, *compilation*, *execution*, and *interaction*. The SEEDX emulator provides Software Development Kit (SDK) to manage the composition and compilation, while leveraging modern container orchestration frameworks (e.g., Docker and Kubernetes) to seamlessly handle the execution phase.

The Composition Phase (Execution Agnosticism). The objective of the *composition* phase is to define the network

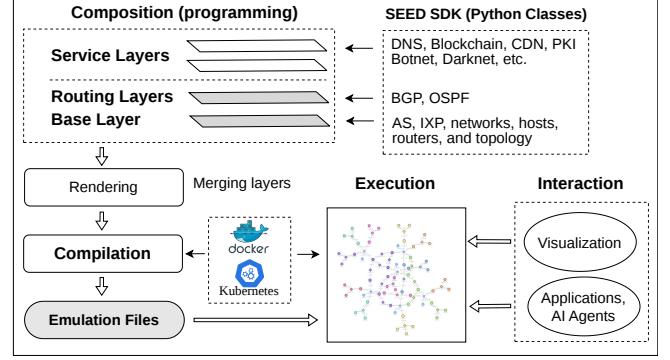


Figure 1: The architecture of the SEEDX Emulator.

topology and provision the constituent nodes, encompassing node instantiation, software installation, and routing initialization. An important design decision to make is what language we should use to conduct the composition. There are three typical approaches, and their qualitative comparison is depicted in Figure 2. The first is to use a graphical language, i.e., drag-and-drop GUI tools. It is used by CORE [4] and GNS3 [24]. The second is to use declarative language, such as YAML and XML. It is used by Mini-Internet [25], Kathara [10], and Containerlab [20].

While these two approaches are quite intuitive to use, they come at a severe cost to expressive power: they struggle with dynamic topology generation, conditional logic, and parameterized scaling. For instance, generating 1,000 nodes with incrementally varying BGP configurations is highly cumbersome using a GUI or static XML file, whereas it requires merely a simple for loop in a programming language. Therefore, given the sheer scale and complexity required for Internet-level emulation, the third approach, using a programming language is a better choice. This approach is used by Mininet, Kathara, Cisco Modeling Lab, and also by our emulator.

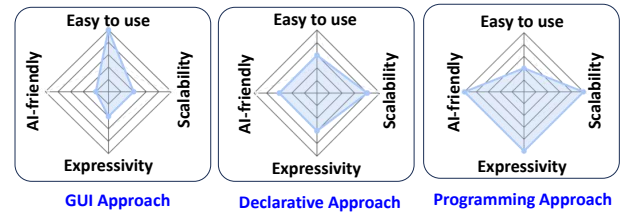


Figure 2: Three typical approaches for composing emulation

We developed an open-source Python library that offers object-oriented abstractions for essential Internet elements. These elements range from core infrastructure components (e.g., Autonomous Systems, subnets, hosts, routers, BGP routers, and Internet Exchange Points) to complex, higher-layer distributed services (e.g., DNS system, botnets, darknets, CDN, and blockchain networks). Utilizing these programmatic building blocks, users can expressively define both the structural topology and the internal software configurations

of the nodes. APIs are provided for users to customize these elements. Appendix A provides code snippets illustrating how to compose an emulated Internet with the SEEDX library.

The programmatic foundation has yielded a profound, unexpected advantage in the era of generative AI. While autonomous coding agents were not a consideration during the emulator’s initial development, our code-centric approach is inherently AI-friendly. Because the emulation is defined using standard Python syntax and well-structured APIs—rather than GUI formats or rigid markup languages—Modern Large Language Models (LLMs) can effortlessly read, reason about, and autonomously generate complex network configurations, drastically lowering the barrier to entry for experiment design.

Another important design decision for the composition phase is the use of layered design, which is essential for extensibility. We will describe this design in details later in Subsection 3.2. The final step of the composition is rendering, which merge all the layers together into an intermediate, technology-neutral representation of the Internet topology and system configuration. The entire composition phase is strictly agnostic to the underlying execution technology.

The Compilation Phase (Execution Dependent). The Compilation phase is responsible for translating the output of the composition—the intermediate representation—into technology-specific execution files. Because the emulator’s composition is strictly decoupled from the execution environment, the compilation phase allows researchers to write their experiment code once and deploy it across different emulation environments. This separation of concerns guarantees the longevity and extensibility of the emulator. For example, when we scale the emulation from a single local machine (based on the Docker technology) to a distributed cloud environment, we only need to implement a Kubernetes compiler, without making any change to the composition.

Currently, we have implemented two compilers, Docker and Kubernetes. The Docker compiler is for emulations running on a single machine. Its primary objective is to translate the emulation into a unified `docker-compose.yml` file and the docker images for each physical node of the emulator. The Kubernetes compiler is for distributed emulations. It translates the exact same emulation into a suite of distributed cloud-native manifests, allowing the emulation to be conducted on a cluster of computers using the Kubernetes technology.

The Execution Phase. The files generated by the compiler are given to the corresponding execution engines, which build the container images and start the emulation. For large-scale emulation, this is quite challenging. Section 5 provides the details of these challenges and how we overcome them.

The Interaction Phase (applications, visualization, and AI agents). Once the emulation starts, users can interact with the live emulator through multiple modalities: manually via

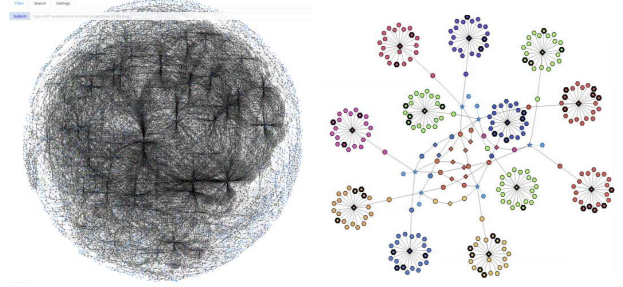


Figure 3: InternetMap: The image on the left shows a visualization of a large-size network using the real-world topology data; it includes 9955 router nodes. Users can zoom in to see details. The image on the right shows the spreading of the worm in a small-size network; the nodes with the bold black border are infected nodes, and they actually flash in the visualization.

standard container command-line interfaces, programmatically using our supplementary Python interaction library, or autonomously via the AI-driven agents. How to enable AI agents to effectively interact with the emulator is an interesting direction that we are currently pursuing; it is beyond the scope of this paper. We have also built several visualization tools for this phase, including a tool called InternetMap, which is a web-based application that graphically displays the network topology and node information, shows packet flows, and allows users to control the emulator (e.g., entering a container to run commands, disabling/enabling BGP sessions, etc.). Figure 3 shows two screenshots of this tool.

Beyond application-level interaction, the platform provides deep access to the emulated data plane via the Linux Traffic Control (tc) utility. This allows users—or automated agents—to dynamically inject network impairments such as latency, jitter, packet loss, and bandwidth constraints into specific links while the emulation is running. By manipulating the kernel’s queuing disciplines (qdiscs) in real-time, researchers can stress-test protocol resilience under adverse conditions or simulate “what-if” scenarios, such as a sudden degradation in a cross-continental fiber link. This capability is fully integrated into our interaction library.

A significant advantage of the SEEDX emulator is its reliance on production-grade software rather than simplified models. This native compatibility allows us to leverage a vast ecosystem of existing open-source and commercial tools without modification. For instance, in our blockchain emulation, we utilize the standard MetaMask wallet [16] for account management and transaction signing. Similarly, we integrate an open-source Beaconchain Explorer [9] to provide real-time visualization of Ethereum blocks, transactions, and validator states. By supporting these off-the-shelf tools, SEEDX provides a high-fidelity environment where users can apply the same tools and workflows they use in real-world deployments.

3.2 Composition: The Layered Design

To manage the inherent complexity of large-scale Internet emulation, the SEEDX Emulator employs a highly modular, decoupled layered architecture, which is inspired by the layering method adopted by most photo editing software, such as Photoshop. This design separates the physical/link-layer topology from the control-plane routing protocols and the application-plane services. This separation of concerns ensures that researchers can independently modify, export, and reuse individual layers without needing to reconstruct the entire experimental environment. The architecture is composed of three primary layer categories (see Figure 1):

The Base Layer (Infrastructure Topology). The Base Layer serves as the foundational substrate of the emulation. It defines the purely structural elements of the network, including the instantiation of Autonomous Systems (ASes), Internet Exchange Points (IXPs), physical nodes (routers and hosts), and the networks connecting them. At this layer, no routing logic or application state exists; it merely represents the logical “bare metal” and cabling of the emulated Internet.

The Routing Layers (Control Plane). Operating directly above the Base Layer, the Routing Layer dictates the network’s control plane. By strictly decoupling the routing logic from the underlying physical or virtual topology, this architecture allows researchers to seamlessly overlay entirely different routing paradigms onto the same Base Layer to conduct comparative studies (as demonstrated by the SCION use case in Section 7.1).

The primary function of the Routing Layer is to act as a configuration compiler: it automatically translates the high-level, user-defined network topology into concrete, protocol-specific configuration files for the underlying routing daemons. In a large-scale emulation, manually configuring routing tables and neighbor relationships is highly error-prone and severely limits scalability. To solve this, the Routing Layer parses the Base Layer’s abstractions—such as Autonomous Systems (ASes), subnets, and link properties—and systematically generates the requisite routing state. This includes automatically establishing eBGP peering across inter-domain links, configuring full-mesh or route-reflector iBGP topologies within an AS, defining OSPF areas for intra-domain reachability, and configuring baseline prefix advertisement and redistribution rules.

Currently, the Routing Layer’s execution backend is powered by BIRD, a high-performance, open-source dynamic routing daemon designed for Unix-like systems. BIRD provides a highly scalable platform for managing BGP, OSPF, and RIP protocols through a dedicated, policy-driven configuration language. While our Routing Layer automates the dense boilerplate required by BIRD to achieve baseline network convergence, it also exposes programmable hooks. These hooks allow researchers to easily inject custom routing

policies into specific nodes—such as BGP route hijacking, path prepending, or traffic engineering rules—without needing to manually rewrite the entire configuration file.

To support a broader ecosystem of network research and ensure the framework remains software-agnostic, future iterations of the Routing Layer will introduce pluggable backends for other industry-standard routing suites, including FRRouting [44] and GoBGP [23].

The Service Layer (Application Plane). The uppermost architectural tier is the Service Layer, which encapsulates the deployment and configuration of applications on top of the Base Layer. Each supported service is implemented as an independent, pluggable module that exposes a high-level API for user customization; it translates user requirements into concrete, software-specific configuration artifacts and command-line execution parameters.

By abstracting away the dense, low-level operational boilerplate, the service layer shields researchers from the historically error-prone and labor-intensive processes of manual software orchestration. This abstraction is particularly critical for Internet-scale emulations, where deploying a single complex service often demands the globally coordinated configuration of hundreds of inter-dependent servers.

Our extensible architecture facilitates the seamless integration of new applications into the emulator’s ecosystem. Leveraging this design, SEEDX has enriched the ecosystem with a series of new components, as listed below: DNS & DNSSEC, Blockchains (including Monero, Ethereum, Chainlink), InterPlanetary File System (IPFS), Public-Key Infrastructure (PKI), Botnets, Content Delivery Networks (CDNs), and Darknets (Tor).

A rich set of service components within the ecosystem is a key feature that most existing emulators do not provide out of the box. Recently, with the help of AI, we are significantly picking up the speed in adding more service components to the SEEDX ecosystem. The more we provide, the easier it is for researchers to conduct their scientific experiments on the SEEDX platform.

3.3 Virtual Nodes and Late Binding

The service layer will not be portable without an important design feature used by the SEEDX architecture: it is the *virtual node binding mechanism*. The SEEDX architecture mirrors hardware design: the base layer acts as a host motherboard, while the service-layer applications function as modular integrated circuit (IC) chips. Rather than hard-wiring application logic directly into the structural blueprint, applications are assigned to symbolic string identifiers (*virtual nodes*) rather than explicit IP addresses or hardware instances. During the Rendering stage, all layers are merged and a binding process resolves these virtual nodes, effectively “plugging” the abstract services into the concrete physical nodes instantiated

by the base layer. This late-binding approach is critical for maintaining the portability and reusability of service layers across vastly different topologies.

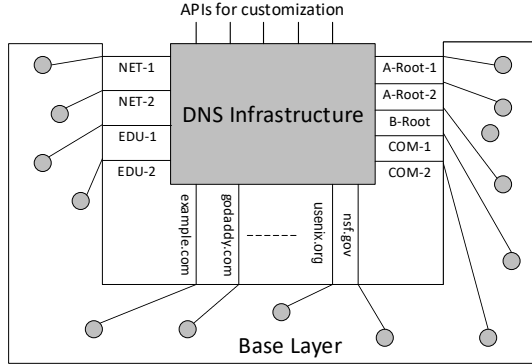


Figure 4: Examples: DNS component

We demonstrate this binding process using a DNS component as an example (see Figure 4). The primary objective of this component is to orchestrate a complete DNS infrastructure, automating the configuration of root, top-level domain (TLD), and authoritative nameservers. When constructing this module, each required nameserver is represented by a Virtual Node. Returning to our analogy, each Virtual Node acts as an exposed “pin” on the DNS chip.

To deploy this self-contained DNS infrastructure within an emulated network, a user simply defines how each pin interfaces with the Base Layer motherboard. For instance, a user might bind the `EDU-1` pin to a randomly selected physical node within Autonomous System 1192, while wiring the `nsf.gov` pin directly to a specific node with the IP address `128.150.4.107`. Because the component’s internal logic is entirely decoupled from the underlying network state, migrating this complex DNS infrastructure to a completely different emulator or topology requires zero reconfiguration of the DNS software itself; the user merely updates the pin bindings. A complete code example is given in Appendix A.6.

The strictly layered architecture of the SEEDX Emulator provides paramount extensibility by fundamentally decoupling the underlying physical network topology from the control plane and application logic. This modular design circumvents the limitations of monolithic emulators, enabling orthogonal extensions—such as integrating novel routing protocols or deploying complex application services—without disrupting adjacent systems. By utilizing abstract “Virtual Nodes” and late binding, developers can encapsulate new service behaviors without hard-coding infrastructural dependencies, making the addition of custom features exceptionally intuitive. Ultimately, this decoupled approach fosters a highly efficient, “composable” ecosystem for scientific research, allowing teams to seamlessly share and combine modular artifacts to rapidly construct complex, multi-domain experimental environments.

Historically, evaluating a novel network architecture is

severely hindered by the “application gap”—researchers can build a new network control plane, but they lack the complex, realistic application-layer traffic needed to test it at scale. Because the SEEDX emulator isolates application logic from the underlying topology via the Virtual Node binding mechanism, research teams can gain instant access to a rich ecosystem of distributed applications. They can seamlessly overlay complex infrastructures—such as distributed DNS hierarchies, Ethereum blockchain networks, or other large-scale system, directly atop their experimental network.

4 High-Fidelity Internet Topology

The fidelity of SEEDX is fundamentally rooted in the structural realism of its underlying Internet topology. While the system maintains support for traditional synthetic graph models, it distinguishes itself by facilitating the construction of emulated environments derived directly from empirical Internet measurement data.

4.1 Real-World Topology Construction

We design an automated topology-generation pipeline that extracts Internet topology information and inter-AS peering relationships from CAIDA datasets [11, 12] and translates them into deployable emulator abstractions. Specifically, the pipeline constructs the structural graph of Autonomous Systems and their connectivity, and automatically derives the corresponding routing policies from the observed peering information, making the resulting emulated network more faithful to the real Internet in both structure and inter-domain routing semantics.

4.2 BGP-Preserving Topology Reduction

Although we can construct a full-scale backbone network topology comprising a massive number of nodes, practical large-scale network emulation scenarios—often constrained by single-machine computational power, memory, or underlying operating system resources—typically require only a representative subset containing thousands of nodes. Therefore, the primary challenge in high-fidelity emulation construction is to extract a representative subgraph from a massive global topology that satisfies experimental scale requirements while maximally preserving the original network connectivity and routing influence.

In traditional graph theory or social networks, physical connectivity between nodes generally implies communication reachability. However, in real-world Internet scenarios governed by commercial policies such as BGP’s valley-free routing, even if AS_1 and AS_2 are physically connected to the same IXP, traffic cannot be exchanged unless a logical peering relationship is established on the control plane. This fundamental

difference necessitates the introduction of protocol-level semantic constraints into topology sampling.

Rather than directly searching for a budget-constrained representative subgraph, we reformulate topology reduction as a seed-selection problem. Specifically, we seek a set of nodes whose routing propagation, under Internet-specific protocol constraints, can cover the most representative portion of the original topology. By maximizing the propagation scope under strict protocol-level constraints, the derived seed set deeply integrates both the physical topological structure and complex routing semantics. As a result, the subgraph constructed through its forward propagation achieves higher fidelity, preserving a much richer set of structural and control-plane characteristics from the original network.

Based on this reformulation, we define the Internet Topology Influence Maximization (ITIM) problem as follows:

Definition 1 (ITIM) *Given an Internet topology graph $G = (V, E)$ and a budget k , the ITIM problem asks to find a seed set $S^* \subseteq V$ of size k that maximizes the expected routing influence spread under the Internet-specific IC model, i.e.,*

$$S^* = \arg \max_{S \subseteq V, |S|=k} \sigma(S).$$

To efficiently solve ITIM, we propose a Subgraph Generation based Influence Maximization (SGIM) algorithm. SGIM cannot directly apply the traditional IMM algorithm [43]; instead, it incorporates BGP routing export policies into the Reverse Reachable set (RR-set) generation process. Specifically, the algorithm first computes and selects the k router nodes with the greatest routing propagation influence in the full topology graph as the seed set. It then performs forward propagation sampling from these seed nodes, selecting the n most frequently activated nodes to form the target-scale subgraph topology. At the same time, the algorithm strictly enforces the microscopic integrity of Autonomous Systems (ASes): once a router node is selected, the remaining internal nodes of its corresponding AS, together with any strongly associated IXP nodes, are mandatorily included in the subgraph as rigid dependencies, thereby ensuring the structural authenticity of the topology slice.

5 Scaling the Execution

Leveraging its native support for distributed orchestration via Kubernetes, SEEDX inherently scales across heterogeneous clusters. However, isolating the emulation to a single physical host provides a critical benchmark for identifying the fundamental performance ceilings of the underlying system. This vertical scaling analysis reveals significant bottlenecks within the Linux networking stack—most notably the serialization of network namespace operations due to contention on the global Routing Netlink (RTNL) lock.

5.1 System Resource Limitations

The first scalability barriers arise from conservative system-wide limits on network state and process concurrency. Default Linux settings for ARP caches (`gc_thresh`) and connection-tracking tables (`nf_conntrack`) can lead to neighbor-table overflows and packet loss when large numbers of containers are interconnected. Similarly, default limits on global file descriptors (`nofile`) and the maximum number of tasks (`TasksMax`) often cause startup failures during highly concurrent deployments.

We address these limits by increasing kernel networking capacities and file-handle thresholds to the million scale, which removes the primary barrier to high-density container instantiation. However, eliminating these system limits does not resolve the main scalability problem. The more persistent bottleneck is the Routing Netlink (RTNL) interface, which is the kernel’s synchronization point for networking operations such as route updates and interface state changes. Although containers isolate network namespaces, they still share the same underlying kernel. As a result, concurrent networking operations across containers are serialized by the global `rtnl_lock`, which becomes a major bottleneck at scale.

5.2 The RTNL Contention Bottleneck

In large-scale Internet emulation, we observe a substantial gap between control-plane convergence—driven primarily by BGP and internal routing protocols—and data-plane convergence. For example, in a 3,000-node deployment, control-plane convergence completes in roughly 500 seconds, whereas data-plane convergence requires more than 13,000 seconds. This gap arises because once the router containers begin installing the full routing state into the kernel, they concurrently issue Netlink operations (e.g., `RTM_NEWROUTE`), creating severe contention on `rtnl_lock` [27, 31].

This contention does not remain confined to the data plane; it feeds back to the control plane. When routing processes spend excessive time blocked on lock acquisition, less CPU time remains available for routing daemons to process protocol messages and advance their state machines. If protocol timers expire, additional routing updates will be triggered, which in turn lead to more Netlink operations. The result is a self-reinforcing feedback loop—a *Netlink storm*—that can prevent the system from converging.

Because RTNL is required to preserve kernel networking consistency, removing or weakening the lock is not a viable solution, at least for now. The Linux kernel community is acutely aware of the RTNL contention issue, and there is a massive, multi-year effort underway to reduce its scope, though "removing" it entirely is an incredibly complex task due to how deeply it is woven into the networking stack: thousands of drivers and kernel modules assume that when they are called, the `rtnl_lock` is held, guaranteeing they do not have race conditions [?].

eBPF-based RTNL profiling. To precisely identify the source of RTNL contention and quantify its impact on scalability, we build a low-overhead eBPF-based profiling framework. The framework attaches kprobes to `rtnl_lock()` and `rtnl_unlock()` to measure lock wait time and hold time; it uses helpers such as `bpf_get_current_pid_tgid()` to attribute contention events to specific processes and containers. We further instrument `rtnetlink_rcv_msg()` and parse `nlmsg_type` to distinguish different classes of Netlink operations, such as `RTM_NEWROUTE` and `RTM_DELROUTE`. This framework lets us move beyond coarse end-to-end timing and directly connect kernel-level lock contention to control-plane behavior. In particular, it allows us to determine whether contention is primarily driven by bursty FIB installation, how it worsens with topology size, and whether it further starves user-space routing daemons of CPU time.

Using the RTNL profiling, we study the behavior of *online kernel updates*, where each RIB change is immediately propagated to the kernel FIB. At 3,000 nodes, the average RTNL lock wait time reaches on the order of 10^5 microseconds; beyond 3,000 nodes, the resulting Netlink storms can prevent convergence altogether.

5.3 Deferred Kernel Updates

To mitigate the synchronization bursts caused by online FIB installation, we tried the *deferred kernel updates* approach, which combines the following mechanisms.

- **Decoupled updates.** We explicitly separate control-plane convergence from data-plane convergence. Routing daemons first reach control-plane convergence, and kernel FIB updates are then installed asynchronously at a controlled rate. This reduces the likelihood of large Netlink bursts during periods of intense routing churn.
- **Jittering.** We introduce randomized perturbations into the update schedule across nodes. Because many nodes would otherwise install routes at nearly the same time, synchronized bursts arise naturally. Jittering breaks this alignment and spreads route installation over time, further reducing global lock contention.

Using the eBPF profiling framework described above, we observe that deferred kernel updates substantially reduce RTNL contention. For example, in a 3,000-node deployment, the average RTNL lock wait time drops from 150,859 μ s to 685 μ s, which is sufficient to avoid convergence failure caused by Netlink storms.

These mechanisms reduce contention during the initial convergence time, and they are best suited to experiments that primarily focus on control-plane behaviors. However, the convergence time is still too long. If we need to keep installing the control-plane route changes into the kernel, the solution is still impractical due to the long data-plane convergence time.

5.4 Multi-VM Deployment

To preserve data-plane fidelity while bypassing the serialization bottleneck imposed by the global RTNL lock, the execution environment must be partitioned across multiple kernel instances. We achieve this by horizontally scaling the emulation across multiple virtual machines (VMs), thereby sharding the RTNL synchronization domain—an approach we term *multi-VM deployment*. While initially counter-intuitive due to hypervisor overhead, our analysis reveals a critical inflection point: as the emulation scale expands, the network convergence latency induced by kernel-level lock contention vastly outweighs the static resource costs of virtualization. By strategically sharding the RTNL lock, we trade marginal resource density for orders-of-magnitude gains in convergence performance.

SEEDX implements multi-VM deployment through distributed execution, orchestrated by a custom emulation compiler that we developed for Kubernetes. The compiler automatically decomposes the target topology into disjoint sub-topologies, which are scheduled across a cluster of independent VM worker nodes. Empirically, we find that limiting a single RTNL lock to 1,000 localized containers keeps the frequency of concurrent network namespace operations safely below the kernel’s saturation threshold, effectively eliminating convergence failures. We quantitatively evaluate the advantages of this distributed approach against traditional single-host deployment in Section 6.

6 Performance Evaluation

SEEDX stably deploys and reaches data-plane convergence for topologies with up to 10,000 routing nodes on a single machine, a scale comparable to the router-level topology derived from our CAIDA dataset (15K+ routers). We evaluate SEEDX along four dimensions:

- *Topology preservation:* evaluate how well SGIM retains the structural and routing-semantic properties of the original Internet graph.
- *Single-host initialization overhead:* evaluate the time and memory consumption of the single-host deployment.
- *Comparison between the multi-VM and single-host deployments:* compare the performance of these two different deployment methods.
- *Network properties:* evaluate the latency and throughput of the emulator.

Experiment Setup. All experiments were conducted on a high-density compute node with a total hardware expenditure of approximately \$30,000 USD (as of 2025). This platform features a dual-socket AMD EPYC 9654 configuration, providing 384 logical cores and 1 TB of DDR5 RAM, operating on Ubuntu 24.04.3 LTS (Linux kernel v6.8.0-106). Routing logic within the emulated nodes is driven by BIRD v2.17.2.

Table 1: The representative topology scales used in the evaluation. The sampled graphs preserve the relative distribution of ASes, IXPs, networks, and peering links.

Router nodes	AS	IXP	Network	Peering
1K (1078)	82	198	1065	4571
2K (1897)	143	254	1876	13277
3K (3083)	312	326	3023	49461
5K (4954)	674	412	4763	117661
7K (7001)	949	639	6708	154939
10K (9955)	1651	772	9343	161201

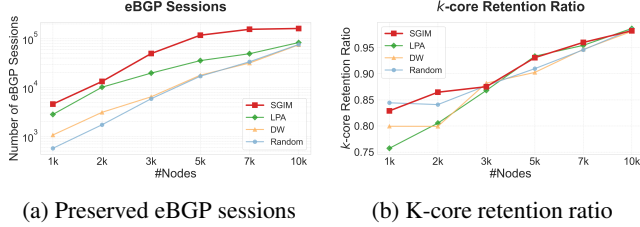


Figure 5: Topology preservation under different target sub-graph sizes. Throughout this section, “#Nodes” denotes the routing-node count.

By employing the SGIM sampling algorithm on the CAIDA Internet topology, we generated experimental datasets ranging from 1,000 to 10,000 routing nodes, as summarized in Table 1. Throughout this evaluation, we adopt shorthand notation (e.g., 3K) for these scales, with exact node counts specified in parentheses within the table.

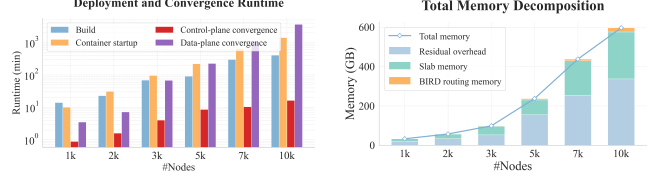
6.1 Topology Preservation

We compare SGIM against three representative baselines: *Random* [26] (uniform sampling with adjacency expansion), *DW* [29] (restart-based random walk), and *LPA* [39] (label-propagation community partitioning). All methods generate subgraphs at identical target sizes and apply the same post-processing to preserve AS-level integrity.

We evaluate preservation quality using two metrics: *K-core Retention Ratio* [40], which captures backbone structural integrity, and the *Number of eBGP Sessions*, which captures control-plane fidelity. Figure 5(a) shows that SGIM preserves substantially more eBGP sessions across all scales, indicating better retention of cross-AS routing relationships than the baseline methods. Figure 5(b) further shows that SGIM maintains a consistently high K-core retention ratio in the medium-to-large regime. Overall, SGIM best preserves both the core graph structure and the routing semantics needed for realistic Internet emulation.

6.2 Single-Host Initialization Overhead

We evaluate the performance of the single-host deployment method, focusing on the deployment cost and memory usage.



(a) Runtime breakdown across deployment stages. (b) Memory usage across different network scales.

Figure 6: Bootstrapping behavior of SEEDX as the network size increases.

Deployment cost. Figure 6(a) breaks initialization latency into four stages. For the convergence stage, we enable *deferred kernel updates* and separately report *control-plane convergence* and *data-plane convergence*. The former denotes the point at which the routing daemons have completed control-plane convergence and selected stable routes, whereas the latter denotes the point at which the corresponding forwarding state has been fully installed into the kernel FIB. The figure shows that the container startup and data-plane convergence increasingly dominate the total cost. Beyond 3K nodes, the sharp growth in data-plane convergence time reflects severe RTNL contention during mass route installation. For control-plane-centric studies, stopping initialization after control-plane convergence provides a practical fast path.

Memory usage. We analyzed the total memory footprint of the emulation environment using host-level metrics from the `free` command. To characterize the memory distribution, we decompose it into three components: user-space routing logic, kernel-level networking structures, and residual container/runtime overhead. The residual term is dominated mainly by per-container runtime state, non-BIRD userspace memory, and filesystem/cache overhead beyond networking-related slab objects.

For the user-space layer, we measured the memory BIRD allocates for the Routing Information Base (RIB) and protocol state using the `"birdc show memory"` command. For the kernel layer, we monitored the data-plane overhead by tracking network-related slab objects via `/proc/slabinfo`. This includes routing-table-related slab objects such as `ip_fib_tribe` and `ip_fib_alias`, as well as packet-buffer-related caches such as `skbuff_head_cache`, `skbuff_small_head`, and `skbuff_ext_cache`.

The residual memory—calculated as the difference between the host’s total used memory and the sum of BIRD and kernel networking structures—is attributed to container isolation overhead (e.g., namespace management and container runtimes). As shown in Figure 6(b), the total memory usage is dominated by container isolation and kernel-space structures rather than the BIRD daemon itself. This suggests that kernel state and virtualization overhead, rather than user-space routing logic, are the primary bottlenecks limiting single-host scalability in the SEED emulator.

6.3 Multi-VM versus Single-Host Deployment

As discussed in Section 5, large-scale route installation and virtual-link creation on a single host are dominated by RTNL contention. To evaluate whether distributing the emulation across multiple kernels alleviates this bottleneck, we compare a 10-VM multi-VM deployment against the single-host deployment. Table 2 summarizes the results.

Initialization overhead. For single-host deployment, *Build* time represents the container image build time, but for multi-VM deployment, *Build* time includes the pods image build time and the time spending on distributing the pods across multiple VMs. Due to the additional step involved, the multi-VM deployment clearly has a higher *Build* time than the single-host deployment. However, such a tradeoff substantially lowers the subsequent container/pod startup time (*Up*), control-plane convergence time (*CP conv.*), and especially data-plane convergence time (*DP conv.*). The advantage widens with scale. At 10K, *Up* time decreases from 85,560 s to 3,468 s, while *DP conv.* decreases from 219,744 s to 915 s. These results indicate that sharding the emulation across 10 VMs substantially reduces the single-kernel bottleneck that constrains large-scale single-host deployment.

Fault recovery. To further evaluate fault recovery at large scale, once the network reaches a stable state, we deliberately terminate the BIRD processes on 10 fixed core routing nodes to emulate routing-node failures, and measure the resulting data-plane reconvergence time (*Reconv.*). Multi-VM deployment yields an even larger advantage during failure recovery. Because route updates and FIB installation are distributed across independent kernels, post-failure reconvergence is accelerated by orders of magnitude. At 10K, *Reconv.* decreases from 189,733 s on single-host to 131 s under the multi-VM deployment. This comparison is conservative in favor of the single-host deployment: in the single-host recovery experiments, we have enabled *deferred kernel updates*; otherwise, recovery fails beyond 5K because of the RTNL-induced update storms. In contrast, the multi-VM deployment uses *online kernel updates* throughout, since it avoids the same degree of RTNL contention. Overall, these results suggest that for large-scale experiments requiring realistic data-plane behavior under dynamic failures, the multi-VM deployment is the more suitable approach.

6.4 Network Properties

We evaluate the emulator’s network properties once it reaches a steady state. Specifically, we measure the round-trip time (RTT) via `ping` and throughput via `iperf3` over randomly sampled 1-hop and 10-hop paths. To simulate realistic conditions, we inject random background flows, ensuring that the measurement and background-flow endpoints encompass approximately 2% of all nodes. As demonstrated in Table 3,

Table 2: Comparison of bootstrapping and recovery time under Kubernetes-based and single-host deployments. All values are reported in seconds. CP conv. and DP conv. denote control-plane and data-plane convergence, respectively.

Scale	Deploy	Build	Up	CP conv.	DP conv.	Reconv.
3K	multi-VM	10548	1278	145	319	18
	single-host	4140	5763	243	4090	771
5K	multi-VM	14818	1706	430	598	33
	single-host	5520	13342	517	13584	11584
7K	multi-VM	33948	2414	594	820	55
	single-host	18120	39113	627	60550	56552
10K	multi-VM	46898	3468	600	915	131
	single-host	24600	85560	986	219744	189733

Table 3: Representative data-plane measurements. The throughput (*Thr.*) value is reported in Gbps, while the RTT value is reported in ms.

#Nodes	1-hop Thr.	10-hop Thr.	Avg. Thr.	1-hop RTT	10-hop RTT	Avg. RTT
1K	17.775	10.780	13.895	0.1335	0.3210	0.2160
2K	16.900	11.387	14.159	0.2110	0.3410	0.2329
3K	16.300	8.005	12.751	0.1420	0.3430	0.2253
5K	15.157	9.047	11.647	0.1274	0.3520	0.2316
7K	13.167	9.700	9.750	0.1996	0.3443	0.3482
10K	10.460	9.300	9.423	0.1367	0.3483	0.3293

SEEDX maintains stable forwarding performance regardless of topology size. Even at a 10K-node scale, the system sustains a 10-hop RTT of approximately 0.34 ms and throughput near 10 Gbps, representing the upper-bound performance of the forwarding plane. For experiments requiring realistic wide-area characteristics, SEEDX can further enforce bandwidth and delay constraints using the Linux traffic control (`tc`) utility.

7 Applications and Case Studies

We use case studies to demonstrate how the SEEDX emulator can be used or has been used in practice. All the experiments that we have conducted can be reproduced. The source code and manuals are placed inside the `nsdi` folder of the project’s public github repository (the `nsdi2027` branch) [2].

7.1 Control-Plane Applications

The SEEDX emulator runs the real routing software in a real operating system, making it well suited for complex control-plane networking and cybersecurity studies.

BGP-related studies. One type of studies focuses on BGP and related control-plane mechanisms. Such studies typically require three key properties from the experimental platform: an AS-level topology that closely resembles the real Internet, real BGP state machines and routing policies, and an environment in which anomalous events can be repeatedly injected at low cost with accurate ground truth. SEEDX satisfies all three requirements. It provides fine-grained BGP observation

and control interfaces, through which, users can access detailed routing state and dynamically influence BGP routing at runtime, including route admission, route propagation, and route attribute manipulation.

To demonstrate how these interfaces support externally driven routing systems, we reproduce a RoST-style case study [5] in SEEDX, focusing on the workflow and capabilities required by RoST rather than its proprietary decision logic. In this setting, routing decisions are made by external logic based on observed routing state and must be enforced through standard BGP routers without modifying the protocol implementation. In RoST, agents—operating outside the routers—observe routing state and derive control actions; in our setup, we realize this workflow by enabling such interactions through the provided interfaces. Concretely, we construct an external control loop in which agents continuously obtain routing state from routers and apply control actions that directly influence routing behavior. The implementation of this case study is included in the `nsdi/rost` folder in the github repository [2].

RoST exemplifies a broader class of inter-domain routing systems that augment BGP with additional logic without modifying the protocol itself; similar ideas have been explored in prior work on BGP observation frameworks such as BG-Stream, programmable routing systems such as xBGP, as well as studies highlighting the limitations of BGP monitoring [8, 36, 46]. Despite differences in design, these efforts share a common requirement: they must be evaluated in environments where routing behavior can be observed and influenced while preserving realistic inter-domain dynamics. Existing approaches only partially satisfy this need. Simulation and trace-driven analysis offer flexibility and low cost, but capture only limited aspects of routing behavior and fail to fully reflect protocol dynamics. In contrast, experiments on real networks provide higher realism, but are difficult to control, reproduce, and scale. SEEDX addresses this gap by constructing networks using real routing software such as BIRD within a unified and reproducible environment. It enables detailed visibility into routing state and fine-grained, programmable control over routing behavior through its observation and control interfaces, while maintaining full control over experimental conditions, thereby supporting systematic evaluation of BGP-based mechanisms under realistic settings.

SCION. Beyond evaluating traditional BGP mechanisms, SEEDX also serves as a robust foundation for evaluating clean-slate architectures. A premier case study demonstrating the platform’s architectural flexibility is the integration of the SCION (Scalability, Control, and Isolation On Next-Generation Networks) project. SCION [6] is a clean-slate Internet architecture, and it inherently bypasses traditional IP routing and standard BGP. To evaluate this novel paradigm, SCION researchers leveraged the SEEDX emulator’s decoupled design to cleanly replace the standard Base and Routing

Layers with a SCION layer, which includes SCION’s custom Isolation Domains (ISDs), Control Services, and Border Routers. This substitution highlights how easily novel “ground-up” architectures can be bootstrapped within the emulator without fighting rigid, hard-coded infrastructural dependencies. SCION layer is already integrated into the SEEDX code base and has been used by researchers beyond the SCION group.

7.2 Service-Layer Applications

The rich set of the service components makes SEEDX suitable for evaluating service-layer applications, especially the Internet-scale distributed systems, where reproducing complex, globally dispersed applications is notoriously difficult.

Blockchain (Ethereum and Monero). We have implemented components for two blockchains: Monero and Ethereum. Using these components, users can easily deploy large-scale private blockchains on top of a realistic Internet topology. These blockchains run the same client programs as those on the mainnet, therefore, all the existing tools can seamlessly interact with our blockchain emulator.

A recent study (currently under submission to a top-tier security conference) utilized the emulator to evaluate a novel, real-time topology discovery technique against the Monero network. Aggressive topology discovery on privacy-preserving blockchains is equivalent to a network attack, so executing such experiments on the live mainnet poses severe ethical risks. By utilizing SEEDX, the researchers safely instantiated 2,000 fully functional Monero routing nodes distributed across 100 emulated Autonomous Systems. Crucially, this deployment represents a significant majority of the roughly 2,500 to 5,000 active, publicly reachable routing nodes currently observed on the live Monero mainnet [13, 32], proving the emulator’s capacity to support production-scale security research within a safely contained sandbox.

In terms of Ethereum, we successfully instantiated a topology of 600 fully functional Ethereum beacon nodes on a single commercial off-the-shelf server (the same as the one used in our evaluation experiments). It is important to contextualize this density within Ethereum’s Proof-of-Stake (PoS) architecture. While the public Ethereum mainnet contains over a million registered validators, there are only 8,000 to 9,000 active beacon nodes [14, 22]. It is these beacon nodes that form the actual P2P infrastructure, route consensus messages and sync the ledger, making them, not the validators, the primary focus of network-layer research. Our 600-node single-server deployment effectively emulates nearly 7% of the global production network. This empirically demonstrates that researchers can conduct Internet-scale consensus experiments on a single machine costing approximately \$30,000 USD, drastically lowering the financial and infrastructure barriers to entry for robust distributed systems research.

CDN: Content Delivery Network. SEEDX is also well suited to CDN research, where experiments require coordinated control over naming, routing, edge behavior, origin interaction, and failure injection. The current version of SEEDX already includes a lightweight CDN implementation that is sufficient to emulate key CDN functions. We use this CDN to demonstrate how the SEEDX emulator can support both security-oriented and systems-oriented CDN studies. We reproduced the core mechanism of the RangeAmp attack in *CDN Backfired* [30], where a small client range request can induce a disproportionately large origin-side transfer. We also reproduced the main qualitative result of *C2DN* [47], where erasure-coded edge storage eliminates post-failure origin fetches that remain unavoidable in a conventional single-copy baseline. The details of the experiments are included in the `nsdi/CDN` folder in the github repository.

Large-Scale Attacks and Defense. Another application of the SEEDX emulator is cybersecurity education platforms. The SEEDX project sparked an initiative named *Yesterday Once More*, aiming to recreating highly profiled large-scale Internet attacks or incidents, bring students back to the past to *experience* historical attacks in person. We recreated a number of attacks, including the Morris worm (1988, the first Internet worm), the Pakistan’s hijack of YouTube (2008, BGP prefix hijacking), and the Mirai botnet attack (2016). With the provided visualization tools, students can observe the technical unfolding of these past events and even interfere to change their course. These recreations are being used in the classroom and exhibition. The demonstration system of these attacks are included in the `tools/DemoSystem` folder.

Small-scale SEEDX emulator is suitable for serving as the platform for hands-on lab exercises. These emulators typically involves a few hundreds of nodes, and can run on a single virtual machine with 2 cores and 4-8 GB of RAM. Several lab exercises have been developed on top of the SEEDX emulator, covering DNS, DNSSEC, BGP, BGP attacks, and Blockchains. They have been deployed via the popular SEED Labs platform [1], which are being used by over 1200 institutes in more than 80 countries. 180 instructors from these institutes have requested the lab manuals for the emulator-based labs.

7.3 Limitations

While SEEDX enables high-fidelity experimentation, its design prioritizes control-plane realism and topological scale over gate-level hardware simulation. Because SEEDX leverages software-based networking and general-purpose CPUs, it does not intend to replicate hardware-specific transients, such as ASIC-level buffer micro-bursting or NIC-offload jitter. Furthermore, while SEEDX can ingest empirical data to reproduce complex phenomena—such as route oscillations or periodic traffic fluctuations—the fidelity of these emulations is ultimately bounded by the granularity of the provided

measurement traces. On the data plane, as with all container-based approaches, fine-grained queuing characteristics and transport-layer timing are subject to the scheduling quanta of the Linux kernel, which may introduce subtle deviations from wire-speed hardware performance at extreme throughputs.

8 Conclusion and Future Work

This paper presents SEEDX Emulator, a programmable Internet emulation platform for both research and education. By decoupling composition from execution and adopting a layered architecture that separates topology, routing, and services, SEEDX provides a flexible, reusable, and execution-agnostic environment for Internet experimentation. Built on real-world topology construction, the BGP-semantic-preserving SGIM algorithm, and a system-level optimization for mitigating RTNL contention, SEEDX achieves an effective balance among fidelity, scalability, and extensibility, and supports high-fidelity single-machine emulation with 10K+ routing nodes. Through a diverse set of case studies, we further show that SEEDX provides a unified, controllable, and repeatable experimental infrastructure for applications spanning control-plane security, clean-slate architectures, blockchain, CDN, and cybersecurity education.

Future Work. Our current large-scale execution model relies on a single physical host partitioned into multiple KVM-based VMs and orchestrated by Kubernetes. While this architecture effectively mitigates single-kernel bottlenecks, virtualization inevitably introduces additional overhead. An immediate direction for future work is therefore to characterize the trade-off between the number of VMs and emulation performance, and to identify operating points that balance scalability against virtualization cost.

While SEEDX leverages Kubernetes for distributed workload scheduling, a fundamental architectural impedance mismatch exists between the design philosophy of cloud-native orchestration and the strict requirements of high-fidelity network emulation. Kubernetes was engineered to abstract away infrastructure, optimizing for the load-balanced deployment of ephemeral, decoupled microservices. In contrast, the containerized nodes in SEEDX are tightly coupled by a strict logical network graph. Because the default Kubernetes scheduler is oblivious to these physical link dependencies, it scatters nodes sub-optimally across the cluster. Consequently, developing a topology-aware scheduling algorithm represents a critical avenue for future work. By ensuring that topologically adjacent nodes are co-located on the same physical worker machines, we can drastically minimize cross-host tunneling overhead, reduce data plane latency, and maximize the overall scalability of the distributed emulation.

References

- [1] SEED Labs: Hands-on Labs for Cybersecurity. Available at https://seedsecuritylabs.org/Labs_20.04/Networking/.
- [2] SEEDX Emulator's Github Repository. The link is omitted due to the blind review requirement; it is known to the program chair.
- [3] The DETER Project. <https://deter-project.org/>.
- [4] J. Ahrenholz, C. Danilov, T. Henderson, and J. Kim. Core: A real-time network emulator. *MILCOM 2008 - 2008 IEEE Military Communications Conference*, pages 1–7, 2008.
- [5] Yosef Edery Anahory, Jie Kong, Nicholas Scaglione, Justin Furuness, Hemi Leibowitz, Amir Herzberg, Bing Wang, and Yossi Gilad. Suppressing {BGP} zombies with route status transparency. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 1349–1366, 2025.
- [6] David Barrera, Laurent Chuat, Adrian Perrig, Raphael M. Reischuk, and Pawel Szalachowski. The scion internet architecture. *Communications of the ACM*, 60(6):56–65, 2017.
- [7] Terry Benzel. The science of cyber-security experimentation: The DETER project. In *Proceedings of the 27th Annual Computer Security Applications Conference (ACSAC)*, pages 137–148. ACM, 2011.
- [8] Henry Birge-Lee, Maria Apostolaki, and Jennifer Rexford. Global bgp attacks that evade route monitoring. In *Passive and Active Measurement*, pages 335–357. Springer, 2025.
- [9] Bitfly. Beaconcha.in: Ethereum beacon chain explorer, 2026. Accessed: 2026-04-20.
- [10] Gaetano Bonofiglio, Veronica Iovinella, Gabriele Lospoto, and Giuseppe Di Battista. Kathará: A container-based framework for implementing network function virtualization and software defined networks. In *NOMS 2018 - 2018 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9. IEEE, 2018.
- [11] CAIDA. The CAIDA AS Relationships Dataset. <https://www.caida.org/catalog/datasets/as-relationships/>, 2016. Date range used: 2024-01 to 2024-12.
- [12] CAIDA. The CAIDA AS Relationships Geolocation Dataset. <https://www.caida.org/catalog/datasets/as-relationships-geo/>, 2016. Files used: 201603.as-rel-geo.txt.gz and 201603.locations.txt.gz.
- [13] Tong Cao, Jiangshan Yu, Jeremie Decouchant, Xiapu Luo, and Paulo Verissimo. Exploring the Monero peer-to-peer network. In *Financial Cryptography and Data Security (FC)*, pages 578–594. Springer, 2020.
- [14] Chainsafe Systems. Nodewatch: Ethereum Consensus Node Analytics. <https://nodewatch.io/>. Accessed: 2026-04-01.
- [15] Cisco Systems, Inc. *Cisco Modeling Labs (CML) Documentation*, version 2.x edition, 2026. Accessed: 2026-04-19.
- [16] ConsenSys. Metamask: A crypto wallet & gateway to blockchain apps, 2024. Accessed: 2026-04-20.
- [17] NS-3 contributors. NS-3: Network Simulator. <https://www.nsnam.org/>, 2022.
- [18] Cyberbit. Cyber Range for Higher Education - Build or Buy? <https://storage.googleapis.com/stateless-www-cyberbit-com-liv/2018/10/Cyber-Range-Build-vs.-Buy-White-Paper.pdf>, 2018.
- [19] CZ.NIC. BIRD Internet Routing Daemon. <https://bird.network.cz/>, 2022.
- [20] Roman Dodin et al. *Containerlab: Orchestrating Network Labs with Docker*, 2026. Accessed: 2026-04-19.
- [21] Wenliang Du, Honghao Zeng, and Kyungrok Won. Seed emulator: an internet emulator for research and education. In *Proceedings of the 21st ACM workshop on hot topics in networks*, pages 101–107, 2022.
- [22] Ethernodes. Ethereum Network & Node Explorer - Consensus Layer Statistics. <https://ethernodes.org/>. Accessed: 2026-04-01.
- [23] Ishiguro Fujisawa et al. Gobgp: A bgp implementation in go. In *Asia Pacific Regional Internet Conference on Operational Technologies (APRICOT)*, 2016.
- [24] GNS3. Graphical Network Simulator-3. <https://www.gns3.com/>, 2022.
- [25] T. Holterbach, T. Bühler, T. Rellstab, and L. Vanbever. An open platform to teach how the internet practically works. *SIGCOMM Comput. Commun. Rev.*, 2020.
- [26] Pili Hu and Wing Cheong Lau. A survey and taxonomy of graph sampling. *arXiv preprint arXiv:1308.5865*, 2013.
- [27] Kernel Newbies. Linux_6.14. https://kernelnewbies.org/Linux_6.14, 2025. Accessed: 2026-03-24; last updated: 2025-06-02.

- [28] B. Lantz, B. Heller, and N. McKeown. A network in a laptop: Rapid prototyping for software-defined networks. In *Hotnets*, Monterey, CA, USA, October 20–21 2010.
- [29] Jure Leskovec and Christos Faloutsos. Sampling from large graphs. In *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 631–636, 2006.
- [30] Weizhong Li, Kaiwen Shen, Run Guo, Baojun Liu, Jia Zhang, Haixin Duan, Shuang Hao, Xiarun Chen, and Yao Wang. Cdn backfired: Amplification attacks based on http range requests. In *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN 2020)*, pages 14–25. IEEE, 2020.
- [31] Linux Kernel Developers. Linux kernel networking documentation. <https://www.kernel.org/doc/html/latest/networking/>, 2024. Accessed: 2026-03.
- [32] Monero.fail. Monero node tracker and network map. <https://monero.fail/map>. Accessed: 2026-04-01.
- [33] NetSim. NetSim Network Simulator. <https://www.tetcos.com/>, 2022.
- [34] NVIDIA Corporation. *NVIDIA Air: Cloud-Native Network Simulation Platform*. NVIDIA Networking, 2026. Accessed: 2026-04-19.
- [35] OpenConfig Project. *KNE: Kubernetes Network Emulation*. Google / OpenConfig, 2026. Accessed: 2026-04-19.
- [36] Chiara Orsini, Alistair King, Danilo Giordano, Vasileios Giotsas, and Alberto Dainotti. Bgpstream: A software framework for live and historical bgp data analysis. In *Proceedings of the 2016 Internet Measurement Conference*, pages 429–444. ACM, 2016.
- [37] Manuel Peuster et al. Containernet: A Docker-based fork of Mininet. <https://github.com/containernet/containernet>, 2015.
- [38] Manuel Peuster, Holger Karl, and Steven van Rossem. MeDICINE: Rapid prototyping of production-ready network services in multi-pop environments. In *2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pages 148–153. IEEE, 2016.
- [39] Usha Nandini Raghavan, Réka Albert, and Soundar Kumara. Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E*, 76(3):036106, 2007.
- [40] Stephen B. Seidman. Network structure and minimum degree. *Social Networks*, 5(3):269–287, 1983.
- [41] Job Snijders, John Heasley, and Martijn Schmidt. Use of BGP Large Communities. RFC 8195, June 2017.
- [42] Gabriel L. Somlo. Greybox: Single-host internet simulator. <https://github.com/cmu-sei/greybox>, 2016.
- [43] Youze Tang, Yanchen Shi, and Xiaokui Xiao. Influence maximization in near-linear time: A martingale approach. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, pages 1539–1554, 2015.
- [44] The FRRouting Contributors. FRRouting (frr) - an open-source routing protocol suite. <https://frrouting.org/>, 2024. Accessed: 2024-xx-xx.
- [45] Wikipedia contributors. Opnet — Wikipedia, the free encyclopedia, 2021. [Online; accessed 9-January-2022].
- [46] Thomas Wirtgen, Tom Rousseaux, Quentin De Coninck, Nicolas Rybowski, Randy Bush, Laurent Vanbever, Axel Legay, and Olivier Bonaventure. xbgp: Faster innovation in routing protocols. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 575–592. USENIX Association, 2023.
- [47] Juncheng Yang, Anirudh Sabnis, Daniel S. Berger, K. V. Rashmi, and Ramesh K. Sitaraman. C2DN: How to harness erasure codes at the edge for efficient content delivery. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1159–1177. USENIX Association, 2022.

A Code Example

We provide a code snippet here to show how to use SEEDX to construct an Internet emulator. More examples can be found from the project’s GitHub repository, inside the `examples` folder.

A.1 Internet Exchange

An autonomous system needs to connect to other autonomous systems, so they can exchange network traffic. Connecting two autonomous systems is called *peering*, which typically occurs inside an Internet Exchange (IX) or colocation center. An IX is basically a big high-throughput switch that connects the routers from different ASes. Through this switch, packets from one AS can be handed over to another AS. In the following example, we create two Internet exchanges, IX-100 and IX-101 (the numbers are the autonomous system number assigned to the IXes). Internally, a network is created for each IX (their network names are automatically set to `ix100` and `ix101` in our convention).

```
ix100 = base.createInternetExchange(100)
ix101 = base.createInternetExchange(101)
```

A.2 Autonomous System

Autonomous system (AS) is an essential element of the Internet. It is emulated in our emulator. When composing the emulator, we can create an autonomous system, and then create hosts, routers, and networks inside it. In the following example, we show how to create a transit AS (with 2 as the AS number).

```
as2 = base.createAutonomousSystem(2)
as2.createNetwork("net0")
as2.createNetwork("net1")
as2.createNetwork("net2")
as2.createRouter('r0').joinNetwork('ix100')
    .joinNetwork('net0')
as2.createRouter('r1').joinNetwork('net0')
    .joinNetwork('ix101')
    .joinNetwork('net1')
as2.createRouter('r2').joinNetwork('net1')
    .joinNetwork('net2')
as2.createRouter('r3').joinNetwork('net2')
    .joinNetwork('ix102')
```

In this example, we first create three internal networks (net0, net1, and net2). We then connect these networks with four routers to form an internal topology; more general topologies can be constructed in the same way. By default, these routers are configured to run the OSPF internal routing protocol. Other routing protocols can also be used. Moreover, the routers r0, r1, and r3 also connect to an Internet exchange network at IX-100, IX-101, and IX-102, respectively. Therefore, these routers are BGP routers. They will be configured to peer with one another via the Internal BGP (IBGP) protocol.

A.3 External BGP (EBGP) Peering

While the IBGP peering is automatically configured by the emulator, for EBGP peering, we need to explicitly specify which ASes peer with each other. In the following example, AS-3 peers with AS-160 and AS-161 at IX-103; their relationship is provider-to-customer, with AS-3 being the provider (i.e., AS-160 and 161 are customers of AS-3; in the real world they typically pay AS-3 for the transit service).

```
ebgp.addPrivatePeerings(103, [3],
    [160, 161], PeerRelationship.Provider)
```

In a public Internet exchange, autonomous systems may peer with many other autonomous systems. To make the peering simple, most Internet exchanges provide a special server called *route server*, so autonomous systems can peer with one another via this server. When the router server receives a route from a participant over BGP, it re-distributes the routes to all other connected participants. In the following example, we peer AS-2, AS-3, and AS-4 at IX-100 using a route server.

```
ebgp.addRsPeers(100, [2, 3, 4])
```

To help implement various peering relationships, we support the BGP Large communities protocol [41] in the emulator. BGP Large Communities have many applications, such as identifying routes by geographic location, encoding business relationships, supporting policy control and many other aspects. In our current implementation, we only use the BGP Large communities to implement the peering relationship, but more interesting applications can be supported by our design.

The routing software used in the SEEDX emulator is called BIRD [19], which is open source. It supports most standard routing protocols. Configuring the BGP and internal routing is quite sophisticated and requires in-depth knowledge of how various routing protocols and how the BIRD software works. Typical users may not have such knowledge. Our emulation library hides all the complexity from the users, who can convey their needs using the provided APIs.

A.4 APIs on Host

To support various applications inside the emulator, we need to be able to install more software and files on the hosts, as well as being able to conduct configuration. While this can be done when the emulator is running, it is time-consuming to do it on many hosts. We therefore provide APIs that allow these tasks to be specified at emulation-construction time. The following example shows how to install software and files on nodes (containers), how to run commands when a container image is built and when the container boots up.

```
host.addSoftware('telnetd')
host.importFile(hostpath="/tmp/f.py", ...)
host.addBuildCommand('useradd -m ...')
host.appendStartCommand('/tmp/server &')
```

With these low-level APIs, users can easily customize nodes. For example, some researchers told us that they need to run a modified Ethereum program *geth* on some nodes. They can easily do that using these APIs. In general, anything that can be done manually on a container can be done through these APIs, which cover all three phases of software: installation, configuration, and execution.

A.5 Hybrid Emulation

By default, the SEEDX Internet emulator is isolated from the outside: machines from outside cannot communicate with those inside the emulator. This closed emulation by itself has many applications, but the scope of the applications can be broadened if we allow a hybrid emulation, i.e., allowing the machines inside the Internet emulator to communicate with those on the real Internet.

With the hybrid emulation, we do not need to duplicate everything inside the emulator. For example, if there is a useful web service that we would like to include in our emulation,

we do not need to duplicate that inside the emulation. Instead, we can access it directly through the hybrid-emulation mechanism.

To allow the emulated Internet to reach the real Internet, we need to direct the packets to an exit point, where the packets can exit from the emulator and enter the real Internet. We use BGP to achieve the goal. We create an autonomous system inside the emulator, and use this AS as the exit point. The BGP router in this AS will announce two prefixes inside the emulator: 0.0.0.0/1 and 128.0.0.0/1. These two network prefixes cover the entire IPv4 address space, so if a destination IP address does not match with any other entry in the routing table, it will match with one of these two prefixes. This makes the exit point a default destination: if a packet does not go to any network inside the emulator, it will be routed towards the exit point. If we only want to reach certain networks in the real Internet, instead of to every network, we can narrow down the scope of the announced prefixes.

The following code snippet shows how to create and set up an exit point inside the emulator. In this example, we create an autonomous system AS-99999, and connect its BGP router to the Internet Exchange ix100.

```
as = base.createAutonomousSystem(99999)
rt = as.createRealWorldRouter('rw',
    prefixes=['0.0.0.0/1', '128.0.0.0/1'])
rt.joinNetwork('ix100', '10.100.0.99')
```

A.6 Component

We use DNS as an example to show how to construct a service-layer component, and how to plug such a component into a base layer.

```
# Create the DNS service layer
dns = DomainNameService()

# Create nameservers for the root zone
dns.install('a-root-server').addZone('.').setMaster()
dns.install('b-root-server').addZone('.')

# Create a TLD nameserver
dns.install('com-server').addZone('com.')

# Create nameserver for second-level zones
dns.install('ns-example-net').addZone('example.net.')

# Add records to the example.net zone
dns.getZone('ns-example-net')
    .addRecord('@ A 1.1.1.1')
    .addRecord('www A 1.1.1.2')
    .addRecord('example.net NS 1.1.1.10')
```

Each of the lines above first create a virtual node, and then install a zone on the node. The same zone can be installed on multiple nodes, as long as one of them is set to Master, so the

others can get the data from the master node. In the final step, we add DNS records to each zone.

The nodes in the DNS component are virtual nodes, representing services. We will install them to physical nodes by binding the virtual nodes to physical nodes. We have created a variety of binding mechanisms. In this example, we only show a simple binding, which uses `Action.FIRST` to bind a virtual node to the first acceptable node (service conflicts can make some nodes unacceptable) that satisfies the filter rule. The following example binds the virtual node `a-root-server` in the DNS infrastructure to the first acceptable node in autonomous system 171.

```
emu.addBinding(Binding('a-root-server',
    filter=Filter(asn=171),
    action=Action.FIRST))
```

A.7 Docker and Kubernetes Compiler

SEEDX provides two primary compilers, one for Docker, and the other for Kubernetes. Compilation is typically performed in the final step, after the emulator has been composed. The purpose of this step is to generate the files for the corresponding execution environment.

```
# Compile the emulation for Docker
docker = Docker()
emu.compile(docker, './output')
```

The Kubernetes execution environment is much more complicated than the Docker environment. When creating the Kubernetes compiler, users can customize the compiler in many aspects, such as scheduling strategy, image pull policy, etc.

```
# Customize the Kubernetes compiler
k8s = KubernetesCompiler(
    scheduling_strategy=scheduling_strategy,
    image_pull_policy=image_pull_policy
    ...
)
# Compile the emulation for Kubernetes
emu.compile(k8s, './output')
```

B Visualization

Being able to visualize what is happening inside the Internet emulator is important. To this goal, we have developed an independent web-based tool for the visualization (called InternetMap). This web application uses the standard docker APIs to retrieve the node and network information from the emulators, and plot them inside a web page. Users can interact with the nodes using the Map tool. When we built the emulator, we added meta data to each container, so meaningful information is preserved and can be retrieved from the emulator using

the docker APIs. Figure 7 shows the GUI interface of the InternetMap.

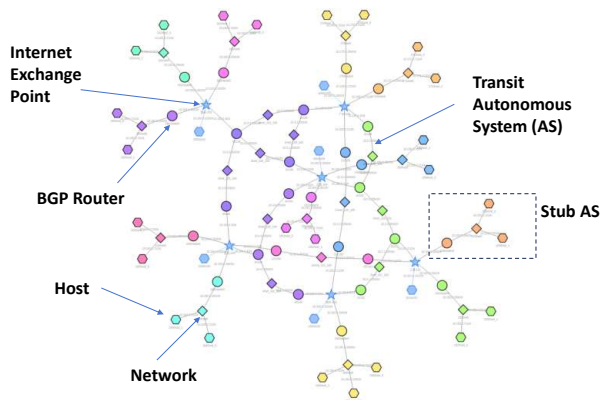


Figure 7: The InternetMap visualization tool

On the InternetMap, using the tcpdump filter expression, users can specify what type of packets they would like to visualize. The tool then requests Docker to run `tcpdump` with the specified filter on all containers. An event will be reported to the InternetMap by a container if it sees a packet matching the filter. The Map will visualize such an event. This way, we can see the packet flow. The tool also supports the record and replay feature, so users can replay events at a slower speed.